



SecretSifter

Engineering Reference & Implementation Memoir

Complete technical record of what was built, how it works, every detection rule, every threshold, every design decision — so you can pick this up 12 months from now and know exactly what you implemented.

VERSION

1.2.27

SOURCE FILES

17 Java files · 13,757
lines

AUTHOR

Hemanth Gorijala

DATE

April 2026

Table of Contents

1. System Overview
2. Build System & Two-JAR Architecture
3. Scan Pipeline — 10 Phases
4. Anchored Vendor Rules (100+)
 - 4.1 Complete Rule Table
 - 4.2 Per-Rule False-Positive Guards
5. Context-Gated Rules (40+)
6. Entropy & Generic KV Scanner
7. False-Positive Mitigations
8. JWT Handling Logic
9. Deduplication Architecture
10. Settings Model & Persistence
11. Threading & Concurrency Model
12. Component Map & Initialization
13. SecretFinding Record & AuditIssue Builder
14. MCP Bridge (JSON-RPC Server)
15. All Hardcoded Thresholds & Limits
16. Test Coverage
17. Version History & Key Changes

1. System Overview

What SecretSifter is, what it does, and the full technical scope of what was built.

SecretSifter is a Burp Suite extension (Montoya API) for passive and active detection of exposed secrets in HTTP traffic. It is not a wrapper around an existing scanner — every detection rule, every false-positive guard, every threshold, and every scan phase was written from scratch.

Metric	Value
Java source files	17
Total lines of code	~13,757
Anchored vendor token rules	100+
Context-gated rules	40+
Scan phases per response	10
Test cases	90+ (PatternsTest + SecretScannerTest)
JAR size (shadow fat JAR)	~430 KB (Gson bundled; Montoya API provided by Burp)
Java version	17 (language target + source compat)
Montoya API version	2026.2
Min Burp version	2024.7
Build time	~12 months active development

Core Capabilities Built

- Passive scanning of all proxy traffic (fires on every response, all tiers)
- Active rescan via context menu (right-click → Rescan for Secrets)
- Bulk Scan tab: URL list, HAR import, 1–50 threads, webpack chunk following
- Headless Chrome integration (via Burp proxy for dynamic SPA traffic)
- Scope Monitor: routes cross-origin proxy findings to the Bulk Scan panel
- Sitemap sweep on load: scans existing site-map responses retroactively

- HTML + CSV + per-domain ZIP report export
- Inline response tab in Proxy History / Repeater
- AI Triage per-finding + bulk (Burp Pro + AI features)
- MCP Bridge: JSON-RPC server on 127.0.0.1:8765 for Claude Code integration
- Full settings persistence to Burp Preferences (`ss.*` keys)
- Two-JAR build: Store (no MCP) and Full (with MCP)

2. Build System & Two-JAR Architecture

Gradle configuration, shadow JAR splits, and the reason for two entry points.

Why Two JARs

PortSwigger BApp Store submission rules prohibit extensions with embedded HTTP listeners or server sockets. The MCP Bridge opens a server socket on `127.0.0.1:8765` for JSON-RPC. To comply with store requirements while shipping the full feature set to direct download users, the build produces two distinct JARs with different entry points:

JAR	Entry Point	MCP Bridge	Distribution
<code>secretsifter-store-*.jar</code>	<code>SecretScannerExtensionStore</code>	Excluded at build time	BApp Store, community
<code>secretsifter-mcp-*.jar</code>	<code>SecretScannerExtension</code>	Included	GitHub Releases, direct download

Gradle Build Tasks

```
./gradlew shadowJar      # store JAR → build/libs/secretsifter-store-*.jar
./gradlew shadowJarFull  # full JAR  → build/libs/secretsifter-mcp-*.jar
./gradlew buildAll       # both JARs
./gradlew test           # JUnit 5 tests
```

Build Configuration Key Points

- `montoya-api:2026.2` is **compileOnly** — Burp provides it at runtime; it must NOT be bundled in the shadow JAR
- `gson:2.10.1` is **implementation** — bundled; used for JSON body walking

- The `shadowJar` task uses `exclude 'com/secretscanner/McpBridge.class'` and `exclude 'com/secretscanner/SecretScannerExtension.class'` to strip the full-build classes
- The `shadowJarFull` task excludes only `SecretScannerExtensionStore.class`
- Both JARs are ~430 KB — the size difference is negligible since `McpBridge` is one class

bapp.json (BApp Store Metadata)

Field	Value
packageName	<code>com.secretscanner</code>
displayName	Secret Sifter
version	1.2.27
minBurpVersion	2024.7
extensionClass	<code>com.secretscanner.SecretScannerExtension</code>
tags	passive-scan, secrets, credentials, api-keys, pii, recon
license	MIT

3. Scan Pipeline — 10 Phases

Every phase that runs when `SecretScanner.scanText()` is called, in order. All phase methods are private; `scanText()` is the only public entry point.

Key design invariant: `SecretScanner` is **fully stateless** — no mutable fields are written during `scanText()`. All patterns are compiled once at JVM load in `Patterns.java` static initializers. Settings are read (not written) via volatile fields or synchronized getters. This makes the scanner safe for concurrent calls from all 50 bulk-scan worker threads simultaneously.

Before phases run, two gates check whether to scan at all:

1. **Content-type gate:** binary, image, video, audio, and font responses are skipped entirely.
2. **HTML inline script extraction:** if Content-Type is HTML, `<script>` blocks (without `src`) and `<script type="application/json">` blocks are extracted and recursively scanned as JS/JSON respectively — preventing double-reporting from the outer HTML body.

1 Anchored Token Scan FAST+

100+ vendor-specific regex patterns with format-anchored prefixes (ghp_, sk-, AKIA, etc.). Near-zero false positive rate because the prefix uniquely identifies the vendor. One regex pass per pattern. Results are **CERTAIN** confidence.

2 URL Credential Scan FAST+

Three patterns: `URL_WITH_CREDS` (https://user:pass@host), `URL_QUERY_CREDS` (?password=X, ≥4 chars), `BARE_QUERY_CREDS` (relative URL, ≥20 chars). Confidence: FIRM.

3a DB Connection String Scan LIGHT+

Two patterns: `DB_CONN_STRING` (mongodb/postgres/mysql/redis/amqp/jdbc URIs with embedded credentials) and `DOTNET_CONN_STR` (SQL Server semicolon format; guarded against arithmetic expressions). Confidence: FIRM.

3b Context-Gated Rules LIGHT+

40+ rules. Each requires a vendor keyword within ~200 chars before the value pattern fires. AWS secret key, Azure DevOps, Snowflake, Algolia, Cloudflare, Auth0, and more. Confidence: FIRM.

4 Generic KV Scanner FULL

Matches key=value and key: value pairs where key is a semantic secret name. Uses `REAL_SECRET_KEYNAME` (100+ key patterns) plus `isProbableSecretValue()` guard. Also runs form-encoded pre-pass for OAuth token responses. Confidence: TENTATIVE.

5 SSR State Blob Unwrap **FULL**

Detects Next.js `__NEXT_DATA__`, Nuxt `__NUXT__`, `window.__INITIAL_STATE__`, `REDUX_STATE`, and similar patterns. Extracts the embedded JSON and re-runs the full pipeline against it recursively.

6 High-Entropy Scanner **FULL**

Shannon entropy $H = -\sum(p_i \cdot \log_2(p_i))$ applied to quoted strings 20–512 chars. Context keyword required within ± 80 chars (± 200 for crypto operations in minified JS). Default threshold: 3.5 bits/char. Confidence: TENTATIVE.

7 Getter Function Scanner **FULL** (JS only)

Two patterns: `GETTER_ARROW_SHORT` (`const getKey = ()  $\Rightarrow$  'VALUE'`) and `GETTER_FUNC_RETURN` (`const getKey = function() { return 'VALUE'; }`). Catches secrets returned from getter methods that are otherwise invisible to key=value scanners.

8 JSON Deep Walk **FULL**

Uses Gson to parse JSON bodies, then walks the tree depth-first. Depth cap: 20 levels. Finding cap: 50 per response (prevents megabytes of JSON flooding reports). Every leaf string checked against `REAL_SECRET_KEYNAME` and scored.

9 XML Leaf Scanner **LIGHT+**

Extracts tag + text content pairs from XML/SOAP responses. Checks tag name against `REAL_SECRET_KEYNAME`. Catches secrets in SOAP web service responses and XML config endpoints.

10 PII Scanner **FULL**

Two sub-phases: (a) SSN — structural format check with exclusion rules (no 000/666/9xx prefix, no 00 middle, no 0000 suffix) + requires SSN context keyword; (b) Credit card — 13–19 digit candidates validated with Luhn checksum algorithm. Confidence: FIRM.

Additional optional phases always available (all tiers):

- **CryptoJS passphrase scan:** `CryptoJS.AES.decrypt("cipher", "KEY")`, `CryptoJS.HmacSHA*(data, "KEY")`
- **Custom rules:** user-defined regex from Settings (runs ALL tiers)
- **Request header scan:** opt-in; checks `KNOWN_SECRET_HEADERS` and semantic key headers

4. Anchored Vendor Rules (100+)

Every rule has a vendor-documented format prefix that makes it format-anchored.
All run at FAST tier and above.

4.1 Complete Rule Table

Rule ID	Pattern Anchor / Format	Rule Name
GITHUB_PAT_001	ghp_[36 a\num]	GitHub PAT (Classic)
GITHUB_PAT_002	gho_[36 a\num]	GitHub OAuth Token
GITHUB_PAT_003	ghs_[36 a\num]	GitHub Actions Secret
GITHUB_PAT_004	ghu_[36 a\num]	GitHub User-to-Server
GITHUB_PAT_005	ghr_[36 a\num]	GitHub Refresh Token
GITHUB_PAT_006	github_pat_[82 a\num]	GitHub Fine-grained PAT
GITLAB_PAT_001	glpat-[20 a\num]	GitLab Personal Access Token
GITLAB_PAT_002	gldt-[20 a\num]	GitLab Deploy Token
NPM_TOKEN_001	npm_[36 a\num]	npm Access Token
SLACK_BOT_001	xoxb-[10-13]-[10-13]-[24 a\num]	Slack Bot Token
SLACK_USER_001	xoxp-[long]	Slack User Token
SLACK_APP_001	xapp-[version]-[ID]-[ts]-[sig]	Slack App-Level Token
SLACK_CONFIG_001	xoxe.[long]	Slack Config Token
SLACK_WEBHOOK_001	hooks.slack.com/services/T.../B.../...	Slack Incoming Webhook
STRIPE_SK_001	sk_live_[24-200 a\num]	Stripe Secret Key (Live)
STRIPE_SK_002	sk_test_[24+ a\num]	Stripe Secret Key (Test)

STRIPE_RK_001	rk_live_[40+ alnum]	Stripe Restricted Key (Live)
STRIPE_PK_001	pk_live_[24+ alnum]	Stripe Publishable Key (Live)
STRIPE_WH_001	whsec_[40 alnum]	Stripe Webhook Secret
SENDGRID_001	SG.[22 alnum].[43 alnum]	SendGrid API Key
TWILIO_SID_001	AC[32 hex]	Twilio Account SID
TWILIO_SID_002	SK/F0/FW/AT + [32 hex]	Twilio API Key SID variants
OPENAI_KEY_001	sk-[48 alnum]	OpenAI API Key (Legacy)
OPENAI_KEY_002	sk-proj-[48-120 alnum]	OpenAI Project API Key
ANTHROPIC_KEY_001	sk-ant-api[2d]-[93+ alnum]	Anthropic API Key
SHOPIFY_AT_001	shpat_[32 hex]	Shopify Access Token
SHOPIFY_SS_001	shpss_[32+ hex]	Shopify Shared Secret
SHOPIFY_CA_001	shpca_[32+ hex]	Shopify Custom App Token
HUBSPOT_001	pat-[2-3 alpha]-[UUID]	HubSpot Private App Token
MAILCHIMP_001	[32 hex]-us[1-2 digit]	Mailchimp API Key
DATABRICKS_001	dapi[32 hex]	Databricks Personal Access Token

GOOGLE_KEY_001	AIza[10+ a\num]	Google API Key
GOOGLE_OAUTH_001	[12d]-[32 hex].apps.googleusercontent.com	Google OAuth Client ID
GOOGLE_OAUTH_002	ya29.[30+ a\num]	Google OAuth 2.0 Access Token
GOOGLE_CS_001	GOCSPX-[28 a\num]	Google OAuth Client Secret
AWS_AK_001	AKIA[16 a\num]	AWS IAM Access Key
AWS_AK_002	ASIA[16 a\num]	AWS STS Temp Access Key
AWS_AK_003	AROA[16 a\num]	AWS Role Access Key
MAPBOX_001	pk.eyJ[20+ a\num]	Mapbox Public Token
MAPBOX_002	sk.eyJ[20+ a\num]	Mapbox Secret Token
PEM_KEY_001	-----BEGIN RSA/EC/DSA/OPENSSH PRIVATE KEY-----	PEM Private Key
VAULT_001	hvs.[90+ base62url]	HashiCorp Vault Service Token
PULUMI_001	pul-[40 hex]	Pulumi Access Token
LINEAR_001	lin_api_[40 a\num]	Linear API Key
NOTION_001	secret_[43 a\num]	Notion Integration Token
NOTION_002	ntn_[38 a\num]	Notion Token (New Format)
NETLIFY_001	nfp_[36+ a\num]	Netlify Personal Access Token

FIREBASE_001	AAAA[7 alnum]:[140 alnum]	Firebase FCM Server Key
AIRTABLE_001	pat[14 alnum].[64 alnum]	Airtable Personal Access Token
WOOCOMMERCE_001	ck_[40 lowercase alnum]	WooCommerce Consumer Key
WOOCOMMERCE_002	cs_[40 lowercase alnum]	WooCommerce Consumer Secret
DISCORD_BOT_001	[23-28].[6-7].[27] base64url segments	Discord Bot Token
DISCORD_WH_001	discord.com/api/webhooks/[17-19]/[60-68 alnum]	Discord Webhook URL
TWITTER_001	AAAA[20+][40-130 mixed]	Twitter/X Bearer Token
NEWRELIC_001	NRAK-[27 UPPER]	New Relic License Key
NEWRELIC_002	NRII-[32 alnum]	New Relic Ingest Key
DYNATRACE_001	dt0[2 alpha][2 digit].[24].[64]	Dynatrace API Token
TELEGRAM_001	[8-10 digit]:AA[33 alnum]	Telegram Bot Token
MAILGUN_001	key-[32 hex]	Mailgun API Key
PAGERDUTY_001	[ru]k.[18 alnum]	PagerDuty API Key
AGE_001	AGE-SECRET-KEY-1[58 bech32]	Age Encryption Secret Key
ALIBABA_001	LTAI[20 alnum]	Alibaba Cloud Access Key
ATLASSIAN_001	ATATT[100-250 alnum]	Atlassian API Token (v1)
ATLASSIAN_002	ATCTT[100+ alnum]	

		Atlassian API Token (v2)
CONTENTFUL_001	CFPAT-[43 a\lnum]	Contentful Personal Access Token
DO_001	dop_v1_[64 hex]	DigitalOcean Access Token
DO_002	doo_v1_[64 hex]	DigitalOcean OAuth Token
DOPPLER_001	dp.pt.[43 a\lnum]	Doppler Service Token
DUFFEL_001	duffel_live_[43 a\lnum]	Duffel API Token (Live)
EASYPOST_001	EZAK[54 a\lnum]	EasyPost API Key
FRAMEIO_001	fio-u-[64 a\lnum]	Frame.io Token
GRAFANA_001	glsa_[32]_[8]	Grafana Service Account Token
GRAFANA_002	glc_[32+ a\lnum]	Grafana Cloud Token
PLANETSCALE_001	pscale_pw_[43 a\lnum]	PlanetScale Password
PLANETSCALE_002	pscale_tkn_[43 a\lnum]	PlanetScale Token
POSTMAN_001	PMAK-[24 hex]-[34 a\lnum]	Postman API Key
PYPI_001	pypi-AgEIcHlwaS5vcmc[50-200 a\lnum]	PyPI API Token
SENDINBLUE_001	xkeysib-[64 hex]-[16 a\lnum]	Brevo/ SendinBlue API Key
SHIPPO_001	shippo_live_[40 hex]	Shippo API Token (Live)
AZURE_CS_001	DefaultEndpointsProtocol=https;AccountName=...;AccountKey=...	

		Azure Storage Connection String
RUBYGEMS_001	rubygems_[48 hex]	RubyGems API Token
HUGGINGFACE_001	hf_[34 alnum]	Hugging Face User Token
GROQ_001	gsk_[52 alnum]	Groq API Key
REPLICATE_001	r8_[40 alnum]	Replicate API Token
XAI_001	xai-[80 alnum]	xAI / Grok API Key
BUILDKITE_001	bkua_[40 alnum]	Buildkite Agent Token
TAILSCALE_001	tskey-api-[32 alnum]	Tailscale API Key
FLYIO_001	FlyV1 [40+ base64url]	Fly.io Auth Token
LANGSMITH_001	lsv2_pt_[47 alnum]	LangSmith API Key
LANGFUSE_001	sk-lf-[UUID]	Langfuse Secret Key
OKTA_001	SSWS [40-48 alnum]	Okta API Token
CIRCLECI_001	ccipat_[40+ alnum]	CircleCI Personal Token
TERRAFORM_001	at.[90+ alnum]	Terraform Cloud Token
SENTRY_001	sntrys_[64+ alnum]	Sentry Auth Token
FIGMA_001	figd_[43 alnum]	Figma Personal Access Token
DROPBOX_001	sl.[130+ alnum]	Dropbox Access Token
SQUARE_001	sq0atp-[22+ alnum]	Square Access Token

CLOUDINARY_001	cloudinary://[6+d]:[20+ a\lnum]@host	Cloudinary URL with Secret
TEAMS_001	webhook.office.com/webhookb2/[20+ a\lnum]	Microsoft Teams Webhook
JWT_001	eyJ[10+].[10+].[10+]	JSON Web Token (JWT)
APIFY_001	apify_api_[40 a\lnum]	Apify API Token
ASANA_001	1/[15-20 digit]:[32 hex]	Asana Personal Access Token
ELASTIC_001	ApiKey [20+ base64]	Elasticsearch API Key
RAZORPAY_001	rzp_live_[20 a\lnum]	Razorpay Live Key
SUPABASE_001	sbp_[40 hex]	Supabase Personal Access Token
BRAINTREE_001	access_token\$production\$[hex]\${hex}	Braintree Production Token
KLAVIYO_001	pk_[34 hex]	Klaviyo API Key
DEEPSEEK_001	sk-[32 lowercase hex]	DeepSeek API Key
TWITCH_001	live_[8-12 digit]_[30-36 a\lnum]	Twitch Stream Key
PAYSTACK_001	sk_live_[40 UPPER]	Paystack Live Secret Key
ONEPASSWORD_001	ops_eyJ[80-500 a\lnum]	1Password Service Account Token
HARNESS_001	pat.[22 UPPER].[24 hex].[20 UPPER]	

		Harness Personal Access Token
SCALINGO_001	tk-us-[48 alnum]	Scalingo API Token
ADAFRUIT_001	aio_[28 alnum]	Adafruit IO Key
SONARQUBE_001	squ_[40 hex] / sqp_[40 hex]	SonarQube User Token
BCRYPT_001	\$2[abxy]\${1-2d}\${[./A-Za-z0-9]{53}	bcrypt Hash (Password Store)
FLUTTERWAVE_001	FLWPUBK_TEST-[32 hex]-X	Flutterwave Public Key (Test)
FLUTTERWAVE_002	FLWSECK_TEST-[32 hex]-X	Flutterwave Secret Key (Test)

4.2 Per-Rule False-Positive Guards

Several rules have additional guards beyond the pattern itself:

Rule	Guard	Why
Discord Bot Token	Each 3-segment value must contain ≥ 1 digit	JS property chains like <code>actions_1.default.fetchSuccess</code> match the 3-segment structure but have no digits in the middle segment
Algolia API Key (context-gated)	App ID rejected if all-lowercase	Real Algolia App IDs are uppercase (e.g. <code>ABCDEF1234</code>); component names like <code>TbBrandAlgolia</code> have mixed case
Azure Storage Connection String	Reject if AccountName= <code>devstoreaccount1</code>	Azurite dev emulator uses a well-known hardcoded credential — not a real secret
Netlify PAT (<code>nfp_</code>)	Check if "netlify" within 60 chars before match	Netlify RUM / Core Web Vitals analytics tokens match the JWT pattern; proximity check confirms intent
PEM Private Key	Requires ≥ 1 base64 body line after the header	Stub comments like <code>-----BEGIN RSA PRIVATE KEY-----</code> alone (without body) are documentation artifacts, not real keys
JWT	Segment 2 must be valid base64url JSON (<code>eyJ</code> prefix)	Some 3-segment URL paths look like JWTs structurally
Bcrypt hash	Cost factor 04–31 only	Prevents matching version strings formatted like bcrypt cost factors
DB connection strings (.NET)	Reject if contains arithmetic expression (<code>=</code> , <code>+</code> , <code>*</code> , <code>/</code>) after password value	Connection string DSL has assignment operators that can false-match math expressions

5. Context-Gated Rules (40+)

These rules require a vendor keyword within ~200 characters of the value. Without the proximity check, the value pattern alone would match unrelated content. All run at LIGHT tier and above.

Keyword Required	Value Pattern	Rule Name	Severity
aws_secret , aws secret	[40 alnum+/-]	AWS Secret Access Key	HIGH
azure , devops	[52 base64]	Azure DevOps PAT	HIGH
snowflake	[32 hex] (near password/token key)	Snowflake Credential	HIGH
jira , atlassian	[24 alnum] (API token format)	Jira/Confluence API Token	HIGH
salesforce , sfdc	[28+ alnum]	Salesforce Access Token	HIGH
datadog	[32 hex]	Datadog API Key	HIGH
heroku	[UUID format]	Heroku API Key	HIGH
launchdarkly	[40 hex]	LaunchDarkly SDK Key	HIGH
messagebird	[25 alnum]	MessageBird API Key	HIGH
splunk	[UUID format]	Splunk HEC Token	HIGH
twilio	[32 alnum] (auth token)	Twilio Auth Token	HIGH
webex , cisco	[long base64url]	Webex Bot Token	HIGH
apim , subscription-key	[32 hex] (4 variants)	Azure APIM Subscription Key	HIGH
algolia	[32 alnum] (uppercase App ID guard)	Algolia API Key	MEDIUM
cloudflare	[40 alnum]	Cloudflare API Token	HIGH
zendesk	[40 hex]	Zendesk API Token	HIGH
zoom	[UUID] / [22+ alnum]	Zoom JWT/OAuth Token	HIGH
intercom	[40+ alnum]	Intercom Access Token	HIGH
auth0	[32+ base64url]	Auth0 Client Secret	HIGH

<code>jenkins</code>	[USER:TOKEN format]	Jenkins API Token	HIGH
<code>drone</code> , <code>droneci</code>	[UUID]	DroneCI Token	HIGH
<code>travis</code>	[22+ alnum]	Travis CI Token	HIGH
<code>vercel</code>	[24+ alnum]	Vercel Access Token	HIGH
<code>freshdesk</code>	[24+ alnum]	Freshdesk API Key	HIGH
<code>monday</code>	[long alnum]	Monday.com API Token	HIGH
<code>coinbase</code>	[long alnum]	Coinbase API Key	HIGH
<code>paypal</code>	[client_id+secret pair]	PayPal Client Secret	HIGH
<code>ibmcloud</code> , <code>apikey</code>	[44 base64]	IBM Cloud API Key	HIGH
<code>cohere</code>	[40 alnum]	Cohere API Key	HIGH
<code>mistral</code>	[32+ alnum]	Mistral API Key	HIGH
<code>spotify</code>	[32 hex]	Spotify Client Secret	HIGH
<code>sonarqube</code>	[40 alnum]	SonarQube Token (context variant)	HIGH
<code>snyk</code>	[UUID]	Snyk API Token	HIGH
<code>wireguard</code>	[44 base64] (private key format)	WireGuard Private Key	HIGH
<code>vault</code> (legacy)	[long alnum]	HashiCorp Vault Token (Legacy)	HIGH
<code>service_account</code> , <code>"type"</code>	GCP JSON key structure	GCP Service Account JSON Key	HIGH
<code>kubernetes</code> , <code>k8s</code>	[long base64]	Kubernetes Service Account Token	HIGH
<code>cyberark</code> , <code>aim</code>	3 fields: AppID, Safe, Object	CyberArk AIM Credential (3 variants)	HIGH

6. Entropy & Generic KV Scanner

How the FULL-tier heuristic scanner works — the part that catches unlisted keys.

Shannon Entropy Formula

$H(s) = -\sum p_i \cdot \log_2(p_i)$ for each unique character in s

where $p_i = \text{count}(\text{char}_i) / \text{len}(s)$

Default threshold: 3.5 bits/char

Range: 0.0 (all same char) to $\log_2(\text{charset_size}) \approx 6.0$ for random base64

Entropy Scanner Gate Conditions (ALL must pass)

1. Value is a quoted string 20–512 characters
2. Value entropy $\geq$ threshold (default 3.5)
3. Context keyword within ± 80 chars OR key name passes `isSemanticSecretKey()`
4. Value is not a UUID (8-4-4-4-12 format) — unless key ends in `Secret` or `Token`
5. Value does not match JWT format
6. Value is not a placeholder (`example` , `xxxx` , `changeme` , etc.)
7. Value character composition is not homogeneous ($>60\%$ same character = reject)
8. Key name is not in FORCED_NOISE_KEYS (90+ exact matches)
9. Key name does not match NOISE_KEYNAMES regex (60+ noise patterns)
10. Value does not start with JS code fragment patterns (`this.` , `)` , etc.)

Semantic Key Analysis — `isSemanticSecretKey()`

A key name must satisfy BOTH conditions:

- **Secret-domain prefix:** starts with or contains one of: `api` , `app` , `auth` , `secret` , `private` , `priv` , `access` , `user` , `account` , `service` , `client` , `oauth` , `db` , `database`
- **Secret-type suffix:** ends with or contains one of: `key` , `token` , `secret` , `password` , `passwd` , `pwd` , `credential` , `cred` , `pass` , `hash`

Examples: `api_key` ✓, `auth_token` ✓, `client_secret` ✓, `alertKey` × (no secret-type suffix), `key` × (no secret-domain prefix)

Generic KV Pattern

```
# Simplified – actual regex includes quote variants and spacing
(?i)([\\w.-]{2,60})\\s*[:=]\\s*"([A-Za-z0-9_+=|?#~@!$%^&*-/]{8,512})["']
```

Only fires when key passes `REAL_SECRET_KEYNAME` (100+ key patterns) and value passes `isProbableSecretValue()`. Confidence: TENTATIVE.

REAL_SECRET_KEYNAME (partial list)

100+ key patterns. Representative sample:

```
api_key, apikey, api-key, access_token, accessToken, auth_token, authToken,
secret_key, secretKey, private_key, privateKey, client_secret, clientSecret,
password, passwd, pwd, bearer_token, bearerToken, app_secret, appSecret,
oauth_token, oauthToken, service_account_key, db_password, database_password,
encryption_key, signing_key, webhook_secret, webhook_token, jwt_secret,
refresh_token, id_token, master_key, admin_password, root_password, ...
```

7. False-Positive Mitigations

Every FP suppression mechanism that was implemented, in order of impact.

Mechanism	Implementation	What It Prevents
Placeholder filter	<code>isPlaceholder(String)</code> — 20+ dummy patterns	<code>example</code> , <code>test123</code> , <code>xxxx</code> , <code>changeme</code> , <code>dummy</code> , <code>your_</code> , <code>insert_</code> , <code>fake</code> , <code>placeholder</code> , <code>YOUR_KEY</code>
UUID/GUID rejection	<code>isAzureGuid()</code> + key check	Azure <code>tenant_id</code> , <code>client_id</code> (UUID format) reported unless key ends in <code>Secret</code> or <code>Token</code>
JWT body suppression	JWT second segment (<code>eyJ...</code>) rejected as standalone match	JWT payload (base64 JSON metadata) not reported as a secret value
Opaque Bearer suppression	<code>hasBearerVendorPrefix()</code> gate	Generic <code>Authorization: Bearer [random]</code> not reported unless value matches a known vendor prefix
Angular/Vue directive filter	<code>FRAMEWORK_ATTR_PREFIXES</code> set + <code>TEMPLATE_EXPR</code> pattern	<code>ng-model</code> , <code>v-bind</code> , <code>wire:</code> , <code>@click</code> , <code>\${{var}}</code> , <code>\$event</code> not treated as keys
Character homogeneity check	>60% same character → reject	<code>AAAAAAAAAA</code> , <code>1111111111</code> , repeated-char CSS tokens
Noise key regex	<code>NOISE_KEYNAMES</code> — 60+ patterns	Keys ending in <code>Type</code> , <code>Error</code> , <code>State</code> , <code>Kind</code> , <code>Version</code> , <code>Name</code> , <code>Id</code> — discriminator enums, not secrets
FORCED_NOISE_KEYS	90+ exact key names	<code>client_id</code> , <code>csrf</code> , <code>sessionid</code> , <code>nonce</code> , <code>redirect_uri</code> , <code>code_verifier</code> etc.

— OAuth flow fields,
not secrets

Luhn validation	Standard Luhn algorithm on CC candidates	16-digit numbers that are not valid credit cards
CC floating-point guard	Check for decimal point context around CC match	<code>1234567890123456.78</code> (float literal) not reported as CC
CDN blocklist	<code>isExternalCdn()</code> — 30+ domains	jQuery, Bootstrap, Google Analytics, Hotjar etc. — open-source libraries with example keys
Blockchain key rejection	Key names: <code>tx</code> , <code>txid</code> , <code>txhash</code> , <code>block</code> , <code>eth</code> , <code>wallet</code>	Transaction hashes, wallet addresses — blockchain identifiers, not secrets
PEM body requirement	Pattern requires ≥ 1 base64 line after header	Stub comments <code>-----BEGIN RSA PRIVATE KEY-----</code> without body
JS code fragment guard	Value starts with <code>this.</code> , <code>)</code> , <code>}</code>	JavaScript expressions inadvertently matching a key=value pattern
Azure emulator guard	AccountName= <code>devstoreaccount1</code> → reject	Azurite dev emulator well-known credential
Semantic key requirement	<code>isSemanticSecretKey()</code> for generic KV	Bare <code>key</code> , <code>alertKey</code> , <code>stateKey</code> rejected as too generic
User key blocklist	<code>isKeyBlocked()</code> — contains-based	User-defined noise patterns: <code>STORAGE_KEY_</code> , <code>NEXT_PUBLIC_</code> , etc.
Minimum value length	Varies per rule (4–8 chars minimum)	Single-word values not worth reporting
Probable value filter	<code>isProbableSecretValue()</code>	Pure-alpha values, all-digits, bare URLs, single English words

Session ID suppression	Key names: <code>sessionId</code> , <code>jsessionId</code> , <code>PHPSESSID</code>	Session identifiers are not secrets (rotated per-request)
Request dedup	<code>seenRequestValues</code> synchronized set	Same hardcoded API key in every outbound request reported only once per session

8. JWT Handling Logic

JWT findings are contextual — the same JWT in a response means different things depending on what was in the request.

Request State	JWT in Response	Reported As	Rationale
Request has <code>Authorization: Bearer [JWT]</code>	Suppress	Not reported	Normal authenticated API call — response JWT is likely a refresh or resource ID, not a leaked credential
Request has non-JWT credential header (<code>X-API-Key</code> , etc.)	Upgrade to HIGH	HIGH / CERTAIN	Pattern matches a token-issuance endpoint — the JWT in the response is the issued token itself
Request is unauthenticated (no credential headers)	Report as-is	MEDIUM / CERTAIN	JWT in response of unauthenticated request may be an exposed credential
Any request	JWT in request header directly	MEDIUM / CERTAIN	JWT in request headers scanned only if request header scanning is enabled

JWT Validation

Pattern: `eyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}`

Additional check: the second segment (`eyJ...` prefix, base64url-encoded JSON header) must decode to valid JSON-like content. Two-segment values (header.payload without signature) are not reported.

9. Deduplication Architecture

Two completely separate dedup namespaces, each serving a different purpose.

Namespace 1 — Request Value Dedup (`seenRequestValues`)

- **Type:** `Collections.synchronizedSet(new HashSet<String>())`
- **Scope:** Session lifetime (cleared on extension unload)
- **Purpose:** Prevents the same hardcoded API key from being reported once for every request that carries it. If a React app sends `X-API-Key: sk_live_xxx` in 200 requests, the finding is reported once.
- **Used by:** Proxy handler path only. NOT used by bulk scan (one-shot, no shared state).
- **Key format:** Matched value string (e.g. `sk_live_abc123`)

Namespace 2 — Sitemap Dedup (`SitemapDeduplicator`)

- **Type:** Two `ConcurrentHashMap<String, Boolean>` instances — `seen` (bulk scan) and `proxySeen` (proxy handler)
- **Scope:** Session lifetime (cleared on extension unload)
- **Purpose:** Prevents `api.siteMap().add()` from creating duplicate AuditIssues for the same finding at the same URL
- **Why two maps:** If bulk scan and proxy handler share one map, a bulk scan finding for a URL blocks the proxy handler from reporting the same finding when the browser later proxies to that URL at a different location. The split ensures independent dedup per scan path.
- **Bulk scan key format:** `'\u0001' + normUrl + SEP + ruleName` (group dedup) + `normUrl + SEP + matchedValue` (per-finding dedup)
- **Proxy handler:** Uses `shouldAddProxy()` → writes to `proxySeen` map only

URL Normalization for Dedup

- Strip `#fragment`
- Strip `?querystring`
- Strip source suffixes: `[HTML]` , `[JS]` , `[JSON]` , `[XML]` , `[REQ-HEADERS]`

10. Settings Model & Persistence

All settings fields, their types, defaults, and persistence keys in Burp Preferences.

Field	Type	Default	Pref Key	Thread Safety
enabled	volatile boolean	true	ss.enabled	volatile read
tier	volatile ScanTier	FULL	ss.tier	volatile read
entropyThreshold	volatile double	3.5	ss.entropy	volatile read
piiEnabled	volatile boolean	true	ss.pii	volatile read
scanRequestsEnabled	volatile boolean	true	ss.scan_requests	volatile read
allowInsecureSsl	volatile boolean	false	ss.allow_insecure_ssl	volatile read
customRulesEnabled	volatile boolean	true	ss.customrules_enabled	volatile read
mcpEnabled	volatile boolean	false	ss.mcp_enabled	volatile read
cdnBlocklist	List<String>	30+ entries	ss.cdnlist	synchronized
keyBlocklist	List<String>	5 entries	ss.keyblocklist	synchronized
keyAllowlist	List<String>	empty	ss.keyallowlist	synchronized
customRules	List<String>	empty	ss.customrules	synchronized

On settings load (`loadFromPreferences()`), the CDN blocklist is merged with the current defaults to pick up new additions added in extension updates. On save (`saveToPreferences()`), all fields written atomically to their respective keys.

Default Key Blocklist (5 entries)

`STORAGE_KEY_, storage_key_, STATE_KEY_, state_key_, NEXT_PUBLIC_,
REACT_APP_PUBLIC_, VUE_APP_PUBLIC_`

Default CDN Blocklist (30+ entries, excerpt)

`cdnjs, jsdelivr, unpkg, ajax.googleapis.com, fonts.googleapis.com, gstatic.com,
stackpath, maxcdn, firebase (analytics), segment.com, googletagmanager.com,
hotjar.com, cloudflareinsights.com, dc.services.visualstudio.com, launchdarkly.com,
sentry.io, bat.bing.com, online-metrix.net, microsoftonline.com, msauth.net,
adobedtm.com, omtrdc.net, medallia.com, akamaiedge.net`

Note: `amazonaws.com` and `cloudfront.net` are intentionally NOT in the CDN blocklist — AWS API Gateway and CloudFront serve real application responses that may contain secrets.

11. Threading & Concurrency Model

How the extension manages concurrent access across all execution contexts.

Component	Thread Context	Concurrency Strategy
<code>SecretScanner.scanText()</code>	Any (proxy thread, EDT, bulk worker, sitemap sweep)	Fully stateless — no shared mutable fields written during scan
<code>ScanSettings</code> — primitives	Multi-threaded reads, rare writes (UI thread)	<code>volatile</code> — guarantees visibility without lock
<code>ScanSettings</code> — collections	Multi-threaded reads, rare writes	<code>synchronized</code> methods (updates are rare; monitor cost acceptable)
<code>seenRequestValues</code>	Proxy threads (concurrent)	<code>Collections.synchronizedSet(new HashSet<>())</code>
<code>SitemapDeduplicator</code>	Bulk workers + proxy threads	<code>ConcurrentHashMap.putIfAbsent()</code> (lock-free)
<code>BulkScanPanel.currentTargetUrl</code>	50 concurrent bulk workers	<code>ThreadLocal<String></code> — each worker has its own copy
<code>BulkScanPanel.hostToTargetUrl</code>	Multiple workers	<code>ConcurrentHashMap</code>
Bulk scan executor	Up to 50 worker threads	Fixed-size <code>ExecutorService</code> , 1–50 configurable
Proxy handler executor	Proxy pipeline thread	Fixed-size background <code>ExecutorService</code> — proxy thread never blocked
Sitemap sweep	Daemon background thread	EDT snapshot via <code>SwingUtilities.invokeLaterAndWait()</code> , then off-EDT processing
MCP Bridge		

Server accept thread + worker threads	Socket server on 127.0.0.1:8765, thread-per-connection
---	---

macOS EDT Fix (Sitemap Sweep)

On macOS, `api.siteMap().requestResponses()` returns an empty list when called from a non-EDT thread (Montoya's macOS implementation backs the site map with a Swing model). The sitemap sweep uses:

```
List<HttpRequestResponse>[] ref = new List[]{List.of()};  
SwingUtilities.invokeAndWait(() → ref[0] =  
    api.siteMap().requestResponses());
```

This snapshots the site map on the EDT, then processes the snapshot off-EDT. The single-element array is used as a mutable carrier between the EDT lambda and the background thread (Java lambdas cannot capture non-final local variables).

12. Component Map & Initialization

Class	Purpose	Key Design Note
<code>SecretScannerExtension</code>	Full build entry point	Wires all components; starts MCP bridge
<code>SecretScannerExtensionStore</code>	Store build entry point	Identical to full, minus MCP wiring; <code>withMcp=false</code> passed to <code>SettingsPanel</code>
<code>SecretScanner</code>	Core stateless engine	All patterns are static finals in <code>Patterns.java</code> ; engine is injected with settings
<code>Patterns</code>	All compiled regex + rule tables	Static initializers run once at JVM load; 100+ <code>AnchoredRule</code> records + 40+ <code>CtxRule</code> records
<code>ScanSettings</code>	Thread-safe settings model	volatile primitives + synchronized collections + Burp Preferences persistence
<code>SecretFinding</code>	Immutable finding record + <code>AuditIssue</code> builder	record type; <code>toGroupedAuditIssue()</code> has 3-arg overload for <code>issueUrlOverride</code>
<code>BulkScanPanel</code>	Bulk Scan UI + orchestration	<code>ThreadLocal</code> URL, <code>ConcurrentHashMap</code> index, <code>ExecutorService</code> pool
<code>SettingsPanel</code>	Settings UI + preference sync	<code>withMcp</code> flag controls MCP section visibility; MCP instructions text hardcoded here
<code>SecretScanCheck</code>	Passive scan check (Pro only)	Registers <code>PER_REQUEST</code> ; gracefully skips on Community Edition (<code>UnsupportedOperationException</code>)
<code>SecretProxyHandler</code>	Proxy response handler	ALL traffic; uses <code>proxySeen</code> namespace; submits to background executor
<code>SecretContextMenu</code>	Right-click rescan	Expands to all site-map entries for selected host
<code>SecretSifterTab</code>	Inline HTTP response tab	Registered via <code>registerHttpResponseBodyProvider</code>
<code>SitemapDeduplicator</code>		

	Two-namespace dedup guard	seen (bulk) + proxySeen (proxy) — separate to prevent cross-contamination
ScopeMonitor	Cross-origin proxy routing	Parses Referer/Origin headers; routes findings to Bulk Scan results table
HtmlReportGenerator	HTML + per-domain ZIP reports	Self-contained HTML; filterable by severity; CSV download button embedded
McpBridge	JSON-RPC server for Claude Code	127.0.0.1:8765; full build only; start/stop on Settings toggle
ToggleSwitch	Custom Swing toggle component	Replaces standard JCheckBox for MCP enable/disable in Settings UI

Initialization Sequence (Full Build)

```
initialize(MontoyaApi api)
|
1. new ScanSettings()                ← model; not yet loaded from
prefs
2. new SecretScanner(settings, logging) ← stateless engine (Patterns
already compiled)
3. api.scanner().registerPassiveScanCheck(
    new SecretScanCheck(...), PER_REQUEST) ← Pro only; silently
skips on Community
4. new SecretProxyHandler(scanner, settings, api)
    api.proxy().registerResponseHandler(...) ← ALL traffic, background
executor
5. new SecretContextMenu(scanner, settings, api)
    api.userInterface().registerContextMenuItemsProvider(...)
6. api.userInterface().registerHttpResponseEditorProvider(
    new SecretSifterTab(scanner)) ← inline response tab
7. new SettingsPanel(settings, api)
    new BulkScanPanel(scanner, settings, api)
    JTabbedPane: [Bulk Scan | Settings]
    api.userInterface().registerSuiteTab("Secret Sifter", tabPane)
8. settingsPanel.loadFromPreferences() ← NOW load prefs into model +
sync UI
    bulkPanel.syncFromSettings() ← restore saved scan tier to
combo box
    bulkPanel.syncHttpClient() ← rebuild HTTP client (insecure
SSL pref loaded)
9. new McpBridge(bulkPanel, logging, prefs)
    settingsPanel.setMcpBridge(mcpBridge)
    if settings.isMcpEnabled(): mcpBridge.start() ← opt-in; only if
saved as enabled
10. Thread sitemapSweep: sleep(1500ms) → invokeAndWait(snapshot) → scan
off-EDT
11. api.extension().registerUnloadingHandler(...) ← clean shutdown
```


Clean Unloading Sequence

unloadingHandler fires:

1. `mcpBridge.stop()` ← close server socket, drain worker pool
2. `bulkPanel.shutdown()` ← stop active scans, interrupt sweep thread, release index
3. `sitemapSweep.interrupt()` ← stop startup sitemap sweep if still running
4. `proxyHandler.shutdown()` ← drain proxy executor
5. `contextMenu.shutdown()` ← release resources
6. `scanner.clearRequestDedup()` ← free synchronized dedup set
7. `SitemapDeduplicator.clear()` ← clear both ConcurrentHashMaps
8. `ScopeMonitor.clearWatched()` ← remove all watched hosts
9. `ScopeMonitor.setCrossOriginFollow(false)`

13. SecretFinding Record & AuditIssue Builder

The data model for a finding and how it becomes a Burp AuditIssue.

Record Definition

```
record SecretFinding(  
    String ruleId,           // e.g. "GITHUB_PAT_001"  
    String ruleName,        // e.g. "GitHub PAT (Classic)"  
    String keyName,         // e.g. "github_pat_classic"  
    String matchedValue,    // actual secret (truncated >80 chars)  
    String severity,        // HIGH / MEDIUM / LOW / INFORMATION  
    String confidence,      // CERTAIN / FIRM / TENTATIVE  
    int    lineNumber,      // 0-based newline count to match position  
    String context,         // ±40 chars around match  
    String sourceUrl,       // URL (may end with " [JS]", " [JSON]" etc.)  
    String targetUrl        // bulk scan root URL; "" for proxy findings  
)
```

AuditIssue Generation — Key Rules

- **Issue title:** Always "[SecretSifter] Secrets / Credentials" — grouped by rule
- **No CRITICAL severity:** Montoya API has no CRITICAL level; downgraded to HIGH
- **Grouped issue format:** HTML table with all findings for the same rule at the same URL
- **Value truncation:** Matched values >80 chars truncated to 77 + "..." in the detail display
- **Response markers:** Byte-level highlighting of matched values — max 50 per issue, skip if response >4 MB, min value length 4 chars
- **3-arg overload:** `toGroupedAuditIssue(group, rr, issueUrlOverride)` — used by bulk scan to file issues at the target root URL (not the JS file URL) so they appear in the site map even when the JS file itself has no site-map entry

14. MCP Bridge (JSON-RPC Server)

How Claude Code and other MCP agents talk to Burp via SecretSifter.

- **Protocol:** JSON-RPC 2.0 over raw TCP
- **Address:** `127.0.0.1:8765` (loopback only — never exposed externally)
- **Auth:** Bearer token in every request; token generated at first start, persisted to Burp Preferences, displayed in Settings UI with "Copy CLI Command" button
- **Threading:** Thread-per-connection; fixed-size executor for request handling
- **Build:** Full build only (`secretsifter-mcp-*.jar`). BApp Store build excludes the class entirely at build time.
- **Activation:** Opt-in via Settings → MCP Bridge toggle. Not started on extension load by default.
- **Claude Code setup:** `claude mcp add secretsifter --transport http http://127.0.0.1:8765/mcp --header "Authorization: Bearer <TOKEN>"`
- **Restart required:** Claude Code must be restarted after `claude mcp add` — MCP servers are only loaded at session start

15. All Hardcoded Thresholds & Limits

Every magic number in the codebase, in one place.

Parameter	Value	Location	Rationale
Entropy threshold (default)	3.5 bits/char	ScanSettings	Empirically chosen; random base64 $\approx$ 6.0, English text $\approx$ 4.0
Entropy threshold range	0.0 – 6.0	SettingsPanel slider	Upper bound = theoretical max for printable ASCII charset
Quoted string min length (entropy)	20 chars	SecretScanner	Shorter strings have unreliable entropy measurements
Quoted string max length (entropy)	512 chars	SecretScanner	Longer values are likely blobs, not keys; cap prevents runaway match
Context window (entropy keyword)	$\pm$ 80 chars	SecretScanner	Covers typical key=value pair in minified JS
Context window (crypto operation)	$\pm$ 200 chars	SecretScanner	Wider for deeply minified CryptoJS call sites
Context extraction (display)	$\pm$ 40 chars	SecretFinding	Enough context to visually verify without overflowing the UI
JSON walk max depth	20 levels	SecretScanner	Prevents stack overflow / infinite recursion on circular refs
JSON walk max findings	50 per response	SecretScanner	Prevents a 10 MB config

			JSON from flooding the results table
Response marker cap	50 per issue	SecretFinding	Too many markers cause Burp UI lag
Response size (skip markers)	4 MB	SecretFinding	Byte-level search on huge responses is slow; skip highlighting
Value display truncation	80 chars → 77 + "..."	SecretFinding	Long values break the AuditIssue HTML table layout
Minimum value length (markers)	4 chars	SecretFinding	3-char values produce too many false marker highlights
Sitemap sweep delay	1500 ms	SecretScannerExtension	Allow Burp's UI + site-map model to finish initializing before snapshot
Sitemap response body minimum	50 bytes	SecretScannerExtension	Skip empty/tiny responses
Sitemap response status gate	200 – 399	SecretScannerExtension	Skip error/redirect responses
Bulk scan thread count (default)	25	BulkScanPanel	Good balance for typical pentest target bandwidth
	1 – 50	BulkScanPanel	Lower for unstable targets; higher

Bulk scan thread count (range)			for fast local networks
Homogeneity rejection threshold	>60% same char	SecretScanner	Reject AAAA... / 111... patterns that pass entropy due to short length
Netlify RUM proximity window	60 chars	Patterns	Tight window — only suppress if "netlify" is immediately before the token
MCP server port	8765	McpBridge	Arbitrary; unlikely to conflict; documented in settings
Discord segment 3 length	exactly 27 chars	Patterns	HMAC-SHA256 digest in Discord's token format is always exactly 27 chars
SSN prefix exclusions	000, 666, 9xx	Patterns (SSN regex)	ITIN starts with 9, never valid SSN; 000 and 666 are unassigned
CC candidate length range	13 – 19 digits	Patterns	Covers Visa (16), Amex (15), Diners (14), JCB (16), Maestro (12-19)
Placeholder string count	20+ patterns	SecretScanner.isPlaceholder()	Comprehensive list of demo/template dummy values

Forced noise keys	90+ exact names	<code>SecretScanner.FORCED_NOISE_KEYS</code>	OAuth/OIDC standard parameter names that are never secrets
----------------------	-----------------------	--	--

16. Test Coverage

What the test suite covers and what test categories exist.

PatternsTest.java — Pattern Matching Tests

Tests that each regex pattern matches (and does not match) expected inputs.

Groups:

Group	Tests	What's Verified
GitHub	6	ghp_, gho_, ghs_, ghr_, github_pat_, invalid lengths
GitLab	3	glpat-, gldt-, invalid prefix
npm	2	npm_ token, non-matching
Slack	5	bot, user, app, webhook, invalid
Stripe	6	live secret, test secret, restricted, pk_live, pk_test, webhook secret
AWS	4	AKIA, ASIA, AROA, invalid (wrong prefix)
Google	3	Alza key, OAuth client ID, invalid
AI keys	8	OpenAI, Anthropic, Groq, xAI, Hugging Face (valid + invalid)
Shopify	3	shpat_, shpss_, shpca_
Messaging	4	SendGrid, Twilio SID, Mailgun, format validation
PEM keys	4	RSA, EC, bare (no body), OpenSSH — tests body requirement
JWT	2	3-segment valid, 2-segment invalid
SSN	5	valid SSN, 000/666/9xx prefix rejection, 00 middle rejection, 0000 suffix rejection
Credit card	4	Visa (Luhn pass), Mastercard, Amex, too short
Noise keys	2	client_id, nonce — must NOT match as secrets
URL/DB creds	4	https://user:pass@host, ?password=x, postgres URI, ADO.NET string
Additional vendors	25+	Discord webhook, Telegram, Databricks, Mailchimp, Supabase, Razorpay, Okta, CircleCI, Sentry, Figma, DigitalOcean, PlanetScale, Postman, and more

SecretScannerTest.java — End-to-End Pipeline Tests

Tests that run through the full `SecretScanner.scanText()` pipeline:

Test	What It Verifies
GitHub PAT in JS body	Phase 1 (anchored) detects ghp_ token in full response body
GitHub PAT in inline script	HTML script extraction + Phase 1 fires on extracted block
AWS Access Key	AKIA prefix detected with correct severity and ruleId
JWT suppression	JWT not reported when Bearer JWT present in same request context
UUID entropy rejection	UUID value not reported from high-entropy scanner
UUID as Azure client_id	UUID allowed when key is client_id in Azure context (isAzureCredentialKey)
Credit card Luhn pass	Valid Visa number reported when PII enabled
Credit card Luhn fail	Invalid CC number not reported even if format matches
Stripe live key	sk_live_ detected, correct severity HIGH
Stripe test key	sk_test_ detected, correct severity LOW
PEM private key	Full key (header + body) detected; stub comment not detected
Noise key suppression	client_id, nonce with high-entropy values NOT reported
OpenAI key	sk-proj- prefix detected correctly

17. Version History & Key Changes

What changed at each significant version. Useful for remembering when a feature or fix was introduced.

Version	Key Changes
v1.2.27	MCP setup instructions updated in SettingsPanel — added "Restart Claude Code" as step 3 and a triage checklist item. SitemapDeduplicator split into two-namespace design (proxySeen + seen) to fix proxy handler findings being blocked by bulk scan dedup. BulkScanPanel issueUrlOverride made unconditional (no longer requires targetRr != null) to fix bulk scan findings not appearing in Target → Site Map.
v1.2.25–26	Site map visibility fixes (two iterations). Context menu + SecretSifterTab registered as new scan entry points.
v1.2.x series	MCP Bridge integration (McpBridge.java). Two-JAR build architecture. AI Triage (per-finding + bulk). ToggleSwitch custom component. Key name blocklist and allowlist. HAR file import. Debug mode. Headless Browse Chrome integration.
v1.1.0 (2026-03-29)	Custom rules run on ALL tiers (previously FULL only). Multi-file import for custom rules. "Enable custom rules" checkbox. Community rules standard file expanded to 200 patterns. Homogeneity check (>60% single char → reject). Session ID suppression. PEM private key body requirement. Opaque Bearer token suppression. Structural key suffix filter (Type/Error/State/Kind). JS code-fragment value guard (this.,), }). 8 community rule FP fixes.
v1.1.66	Last version before v1.2.x refactor. Reference version: confirmed working sitemap findings (SecretSifter issues appearing under host in Target → Site Map). Used as regression baseline for site-map fix.
v1.0.0	Initial release. Core scan engine (anchored tokens, context-gated, entropy). Passive scan check. Proxy handler. Bulk Scan panel. Settings panel. HTML + CSV export. Sitemap sweep.